



رود مصر الرقمية

2025/11/30

وزارة الاتصالات
وتكنولوجيا المعلومات



DEPI Graduation Project Proposal Data Science Track

Network Intrusion Detection System using Machine Learning

Supervisor: Eng/ Ahmed Ayman

Project Team:

- **Team Leader:** Ahmed Mamdouh Sadek
- **Team Members:**

Dina Fawzy Abdelnaby
Mohamed Farag Hamdallah
Pola Mokhtar Sobhy
Mustafa Ahmed Ibrahim

Resources:

1- **GitHub:** <https://github.com/AhmedELNajjar-dev/Depi-Final-Project>

2- **Google Drive :** <https://drive.google.com/drive/folders/1wa5Y5heNduaFH-T2bIkQAkoXlhiy0QCe?usp=sharing>

Table of Contents

1. Chapter 1: Introduction & Problem Definition

- 1.1 Executive Summary
- 1.2 Business Understanding
- 1.3 The Core Problem
- 1.4 Project Objectives

2. Chapter 2: Data Source & Feature Analysis

- 2.1 Dataset Selection (CICIDS-2017)
- 2.2 Attack Taxonomy
- 2.3 Deep Dive: Network Flow Features

3. Chapter 3: Data Preprocessing & Cleaning

- 3.1 Critical Discovery: The "12-Hour Clock" Bug
- 3.2 Deduplication Strategy: Exact vs. Behavioral
- 3.3 Memory Optimization: Downcasting
- 3.4 The Cleaning Pipeline Summary

4. Chapter 4: Exploratory Data Analysis (EDA)

- 4.1 The Class Imbalance Challenge
- 4.2 Attack Landscape Analysis
- 4.3 Network Entity Analysis (Top Talkers & Targets)
- 4.4 Behavioral Fingerprinting
- 4.5 Temporal & Access Patterns

5. Chapter 5: Feature Engineering & Selection

- 5.1 Metadata Removal for Generalization
- 5.2 Label Grouping (Taxonomy Reduction)
- 5.3 Redundancy Elimination
- 5.4 Addressing Class Imbalance: The 80/20 Strategy
- 5.5 Label Encoding

6. Chapter 6: Model Development Methodology

- 6.1 Algorithm Selection: Random Forest
- 6.2 Training Configuration
- 6.3 Hyperparameter Tuning
- 6.4 Feature Selection: The "Top 26" Optimization

7. Chapter 7: Evaluation & Performance Analysis

- 7.1 Evaluation Metrics
- 7.2 Detailed Performance Results
- 7.3 Error Analysis (Confusion Matrix)
- 7.4 Generalization Check
- 7.5 Impact of Feature Selection

8. Chapter 8: Deployment & Future Roadmap

- 8.1 Deployment Architecture
- 8.2 Performance Benchmarks
- 8.3 Limitations
- 8.4 Future Roadmap

9. Chapter 9: Conclusion

10. References

Chapter 1: Introduction & Problem Definition

1.1 Executive Summary:

The rapid digitization of critical infrastructure and personal data storage has made network security a paramount concern for organizations worldwide. As cyber threats evolve from simple script-kiddie exploits to sophisticated, state-sponsored Advanced Persistent Threats (APTs), traditional security measures have become increasingly inadequate.

This project details the development of a next-generation Network Intrusion Detection System (NIDS) utilizing Machine Learning (ML). By leveraging the Random Forest algorithm and the realistic CICIDS-2017 dataset, we have engineered a system capable of detecting both common and rare network attacks with **99.88% accuracy**. Crucially, this system achieves high performance while utilizing a minimized set of 26 features, ensuring computational efficiency suitable for real-time deployment.

1.2 Business Understanding: The Digital Security Challenge

In the current operational landscape, organizations face a dual challenge:

- 1. Ubiquity of Data:** Nearly all sensitive information—ranging from banking details and trade secrets to personal health records—is transmitted over TCP/IP networks.
- 2. Sophistication of Attacks:** Attackers effectively utilize automation, encryption, and distributed networks (botnets) to bypass perimeter defenses.

The financial and reputational cost of a data breach is at an all-time high. Organizations require security solutions that are not just reactive but proactive, capable of identifying malicious intent before data exfiltration or service disruption occurs. The business goal of this project is to reduce the "Mean Time to Detect" (MTTD) threats and minimize the false positive rate that contributes to security analyst burnout.

1.3 The Core Problem: Failures of Traditional NIDS:

Existing Intrusion Detection Systems (IDS), particularly those relying on signature-based detection (e.g., Snort, Suricata in default configurations), suffer from three critical failures:

Failure 1: Blindness to New Threats (The Zero-Day Problem)

Signature-based systems operate on a "blacklist" model. They compare incoming traffic against a database of known attack signatures.

- **The Flaw:** If an attacker modifies a single byte of their code, the signature changes, and the attack passes through undetected.
- **Consequence:** These systems are fundamentally incapable of detecting Zero-Day attacks (never-before-seen exploits) or polymorphic malware.

Failure 2: Alert Exhaustion (Information Overload):

Modern high-bandwidth networks generate terabytes of traffic daily. Traditional anomaly detection systems often struggle to distinguish between unusual-but-benign behavior (e.g., a flash crowd on a website) and actual attacks (e.g., DDoS).

- **The Flaw:** High False Positive Rates (FPR).
- **Consequence:** Security Operation Center (SOC) teams are overwhelmed by thousands of alerts. In the noise, genuine threats are often ignored or acknowledged too late.

Failure 3: The "Needle in the Haystack" (Rare Attack Detection):

In any network, legitimate traffic makes up 99.9% of the volume. Malicious traffic is statistically insignificant.

- **The Flaw:** Statistical models tend to bias toward the majority class. A model can achieve 99% accuracy by simply predicting "Benign" for every packet, effectively ignoring the 1% of attacks.
- **Consequence:** Stealthy, low-volume attacks like **Infiltration** (internal compromise) or low-and-slow **Botnet** command signals are missed because the model prioritizes global accuracy over minority class recall.

1.4 Project Objectives:

To address these failures, this project established four key technical objectives:

1. **Enhance Detection Capabilities:** Utilize Machine Learning (Random Forest) to learn complex, non-linear behavioral patterns, enabling the detection of polymorphic and zero-day threats.
2. **Optimize Computational Efficiency:** Identify the minimum viable set of network features (Dimensionality Reduction) to allow for high-speed inference without sacrificing accuracy.
3. **Ensure Equitable Detection:** Prioritize the detection of rare attack classes (Infiltration, Web Attacks) alongside common ones (DoS), measured via the **Macro F1-Score**.
4. **Real-World Applicability:** deliver a model that is robust, generalizable, and
5. deployable in a production environment.

Chapter 2: Data Source & Feature Analysis:

2.1 Dataset Selection: CICIDS-2017:

The foundation of any ML-based NIDS is its training data. We selected the **CICIDS-2017** dataset, developed by the Canadian Institute for Cybersecurity (University of New Brunswick).

Why CICIDS-2017 over KDD-99/NSL-KDD?

- **Age:** Older datasets like KDD-99 (created in 1999) reflect network traffic patterns that no longer exist (e.g., Telnet exploits, lack of HTTPS). CICIDS-2017 captures modern protocols and behaviors.
- **Realism:** It includes realistic background traffic generated by the B-Profile system (HTTP, HTTPS, FTP, SSH, and email protocols) mixed with diverse attack vectors.
- **Scale:** The dataset contains **2,830,743** network flows (records) collected over a 5-day period, providing statistically significant sample sizes for most classes.

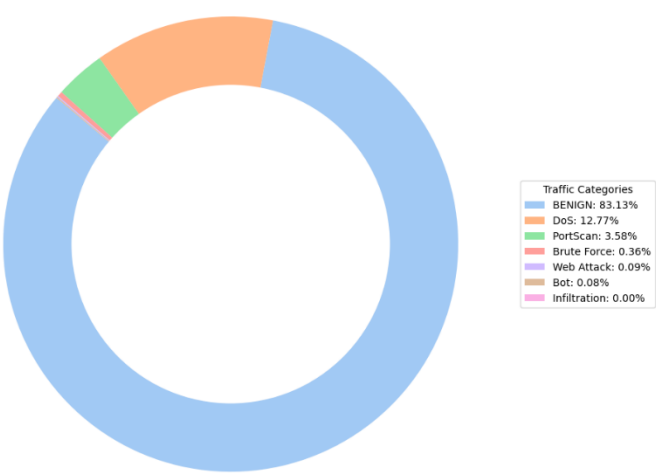
2.2 Attack Taxonomy:

The dataset includes a baseline of Benign traffic and 14 distinct attack labels, which we analyzed and categorized into 7 primary threat groups:

Category	Description	Specific Labels Included	Security Risk
BENIGN	Legitimate user activities.	BENIGN	N/A
DoS	Denial of Service; attempting to exhaust server resources.	Hulk, GoldenEye, Slowloris, Slowhttptest	Service disruption, revenue loss.
DDoS	Distributed DoS; attacks from multiple sources.	DDoS	Massive service outage.
PortScan	Reconnaissance to identify open ports/services.	PortScan	Precursor to specific exploits.
Brute Force	Automated password guessing.	FTP-Patator, SSH-Patator	Unauthorized access/Privilege escalation.

Web Attack	Exploiting vulnerabilities in web applications.	XSS, SQL Injection, Brute Force Web	Data leakage, DB compromise.
Bot	Compromised devices awaiting Command & Control (C2).	Bot	Participation in DDoS, spam, theft.
Infiltration	Exploitation of internal software vulnerabilities.	Infiltration	Deep network compromise, APTs.

Overall Traffic Distribution (Benign vs. Attacks)



2.3 Deep Dive: Network Flow Features:

The dataset provides **79 behavioral features** extracted using the CICFlowMeter tool. These features describe the *metadata* of the connection, not the *payload* (content), which ensures the system works even on encrypted traffic (HTTPS).

2.3.1 Flow-Based Features (Volume & Throughput):

These features measure the sheer amount of data.

- **Features:** Flow Duration, Total Fwd Packets, Total Bwd Packets, Flow Bytes/s.
- **Detection Logic:** High-volume attacks (DoS/DDoS) result in extreme outliers in packet counts and byte rates. Data exfiltration shows abnormally high Bytes/s.

2.3.2 Time-Based Features (Rhythm & Jitter):

These features measure the timing between packets (Inter-Arrival Time - IAT).

- **Features:** Flow IAT Mean, Flow IAT Std, Fwd IAT Min.
- **Detection Logic:**
 - **Human behavior** is bursty and irregular (High IAT variance).

- **Automated attacks** (Bots, Scripted Brute Force) often have machine-like precision with fixed intervals (Low IAT variance).
- **Slowloris** attacks rely on specific timing gaps to keep connections open.

2.3.3 Packet Size Features (The Envelope):

These features analyze the size of the "envelopes" being sent.

- **Features:** Packet Length Mean, Packet Length Variance, Max Packet Length.
- **Detection Logic:**
 - **PortScans** typically use empty or very small packets to ping a port.
 - **Ping Floods** often use fixed-size packets.
 - **Buffer Overflow** attacks may use maximum-sized packets.

2.3.4 Protocol Flags & Headers (TCP Handshake):

These features analyze the control signals of the TCP/IP protocol.

- **Features:** SYN Flag Count, PSH Flag Count, ACK Flag Count, Header Length.
- **Detection Logic:**
 - **SYN Flood:** High count of SYN flags (connection requests) without corresponding ACK (acknowledgments).
 - **Header Tampering:** Anomalies in Header Length can indicate malformed packets designed to crash firewalls.

2.3.5 The "King" Feature: Destination Port:

While technically a metadata feature, the **Destination Port** is the strongest predictor in the dataset.

- **Logic:** Attackers must target specific services.
 - Port 22 traffic is highly correlated with SSH Brute Force.
 - Port 80 traffic contains all Web Attacks.
 - Port 443 traffic contains encrypted/stealth attacks.
- **Significance:** It serves as the primary filter for the model to narrow down the potential attack type.

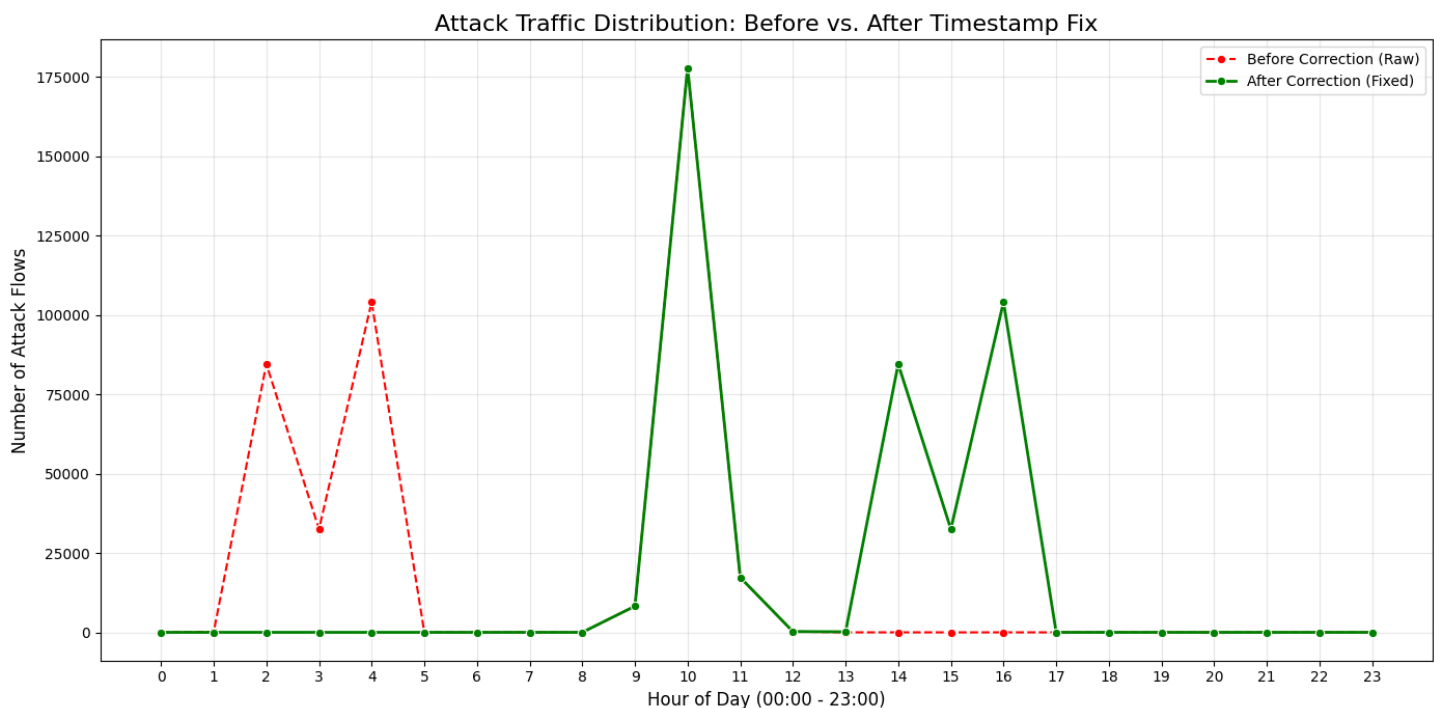
Chapter 3: Data Preprocessing & Cleaning

Data quality is the ceiling of model performance. Given the scale of CICIDS-2017 (2.8 million records), rigorous preprocessing was essential to ensure the model learned actual behavioral patterns rather than artifacts of the collection process.

3.1 Critical Discovery: The "12-Hour Clock" Bug:

During the initial inspection of the raw CSV files, we uncovered a significant anomaly that, if left unaddressed, would have severely impacted any time-based feature analysis.

- **The Symptom:** When plotting traffic volume by hour of the day, we observed traffic *only* between 01:00 and 12:00. There was zero traffic recorded for 13:00 (1 PM) through 23:00 (11 PM), despite the dataset documentation claiming traffic was captured during normal business hours (9 AM - 5 PM).
- **The Root Cause:** The timestamps in the raw CSVs used a 12-hour format but lacked "AM/PM" designators. Consequently, the parsing libraries (Pandas) interpreted 01:00 (afternoon) as 01:00 AM (morning).
- **The Fix:** We implemented a heuristic logic correction. Traffic occurring in the logical "afternoon" block (timestamps 01:00 to 07:59) was shifted forward by 12 hours.
- **Result:** The corrected timeline accurately reflected the traffic capture schedule (9:00 - 17:00), enabling valid analysis of time-dependent attack vectors like brute force campaigns.



3.2 Deduplication Strategy: Exact vs. Behavioral:

The dataset contained 307,078 duplicate rows (approx. 11%). Handling these required a nuanced decision between two deduplication strategies

1. **Behavioral Deduplication (Aggressive):** Removing duplicates based solely on behavioral features (Packet Size, IAT), ignoring IP and Port metadata.
2. **Exact Deduplication (Conservative):** Removing duplicates only if every column, including Source IP and Destination Port, matched.

Decision: Exact Deduplication.

We analyzed the nature of the duplicates and found that PortScan attacks frequently generate thousands of behaviorally identical packets (same size, same flags) sent to different destination ports. Using Behavioral Deduplication would have eliminated ~99% of valid PortScan training data, collapsing it into a single row. By using Exact Deduplication, we removed genuine redundant recordings while preserving the footprint of reconnaissance attacks.

3.3 Memory Optimization: Downcasting:

Loading 2.8 million rows with 79 features requires approximately 2.2 GB of RAM, which creates bottlenecks during iterative training. We implemented a "Downcasting" pipeline to optimize data types without losing information.

- **Integers:** 64-bit integers were inspected. If values fit within smaller bounds (e.g., 0-255), they were cast to int8 or int16.
- **Floats:** Floating-point numbers were maintained at float64 to strictly preserve the statistical precision of variance and mean calculations, which are sensitive to rounding errors.
- **Results:**
 - **Initial Memory:** 2.2 GB
 - **Final Memory:** 1.4 GB (35% Reduction)

3.4 The Cleaning Pipeline Summary:

The final preprocessing pipeline executed the following 10 steps:

1. **Aggregation:** Merging 8 separate CSV files into a single DataFrame.
 2. **Sanitization:** Removal of ~3,000 rows containing NaN or Infinity values caused by division-by-zero errors in flow calculations.
 3. **Metadata Stripping:** Removal of the Flow ID column (a composite string of IP/Port/Protocol) to prevent high-cardinality noise.
 4. **12-Hour Clock Repair:** Timestamp correction.
 5. **Exact Deduplication:** Removal of 307,078 duplicate records.
 6. **Label Normalization:** Standardization of attack names (e.g., merging "Web Attack - XSS" and "Web Attack - Sql Injection" later in the process).
 7. **Data Type Optimization:** Downcasting numeric columns.
 8. **Final Export:** Saving the clean dataset in Parquet format (preserving schema and ensuring 10x faster I/O compared to CSV).
- **Final Dataset Size:** 2,520,158 clean, verified records.

Chapter 4: Exploratory Data Analysis (EDA)

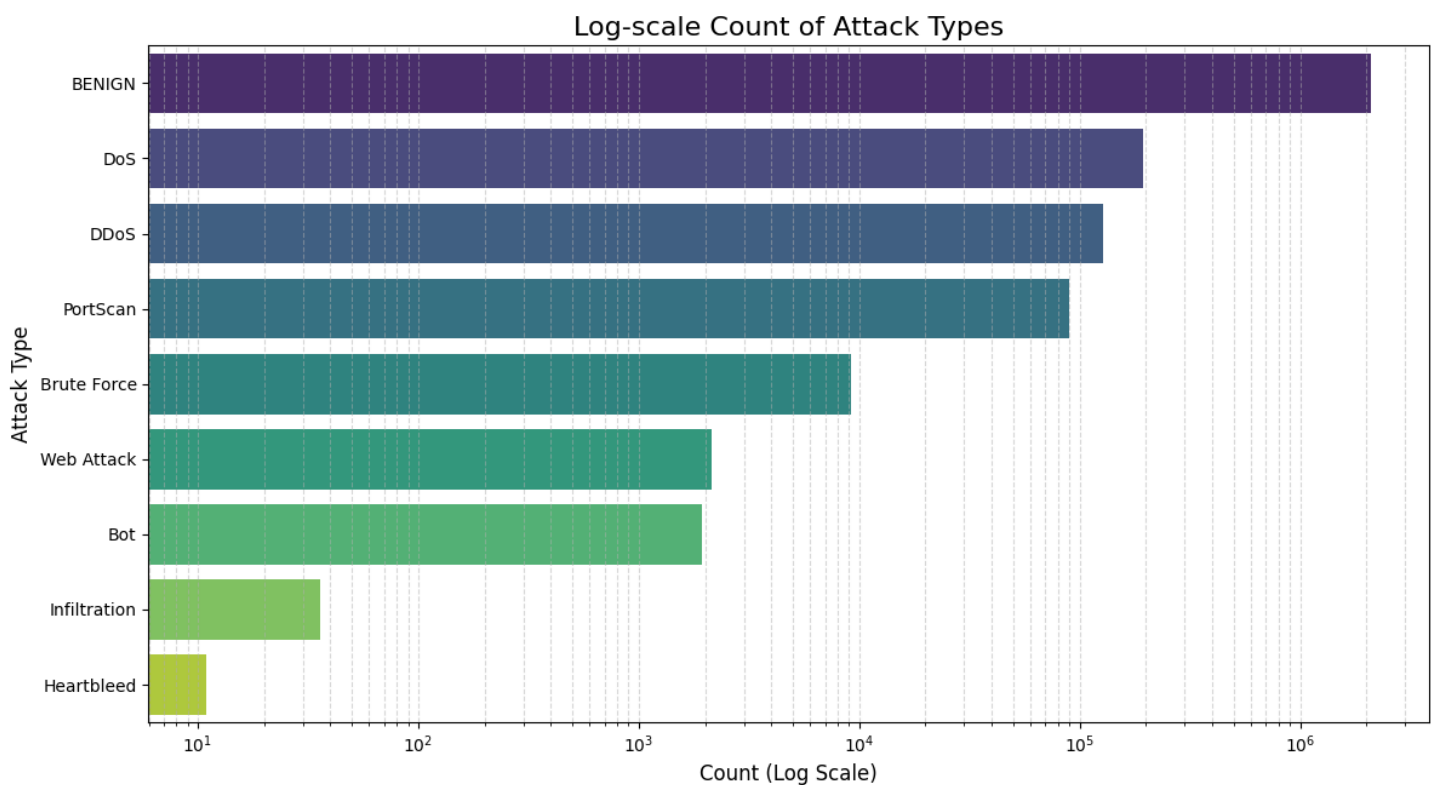
With a clean dataset, we employed a "20 Questions" analytical approach to understand the statistical properties of the network traffic, aiming to uncover specific patterns, anomalies, and signatures for each attack type.

4.1 The Class Imbalance Challenge:

The most immediate insight was the severe class imbalance, a common characteristic of real-world anomaly detection scenarios.

- **BENIGN Traffic:** 2,095,020 records (83.1%)
- **Malicious Traffic:** 425,138 records (16.9%)

Implication: A naive model could achieve 83% accuracy by simply predicting "Benign" for every interaction. This necessitated the use of the Macro F1-Score as our primary evaluation metric and the implementation of a strict 80/20 sampling strategy during feature engineering.



Insight: This log-scale bar chart highlights the vast disparity in frequency between different traffic types.

- BENIGN traffic and DoS attacks dominate the dataset with thousands/millions of instances.
- In contrast, stealthier attacks like Infiltration and Bot activity appear in much smaller numbers (dozens).
- The logarithmic scale is essential here; without it, the bars for the minority classes would be invisible lines.

4.2 Attack Landscape Analysis:

Excluding benign traffic, the threat landscape was dominated by volumetric attacks:

1. **DoS (Denial of Service):** ~45.6% of attacks.
2. **DDoS:** ~30.1% of attacks.

3. PortScan: ~21.2% of attacks.

4. Rare Attacks: Brute Force, Web Attacks, Bots, and Infiltration combined made up less than 3.5% of malicious activity.

4.3 Network Entity Analysis (Top Talkers & Targets):

We analyzed the sources and destinations of the traffic to understand the "Who" and "Whom" of the attacks.

- **Top Talkers (Source IPs):** While many legitimate servers generated high traffic volumes, specific IPs were identified as "bad actors," responsible for thousands of attack attempts. These IPs were distinct from normal high-traffic sources, validating the potential for IP-based reputation filtering (though excluded from our ML model to ensure generalization).
- **Top Targets (Destination IPs):** Attacks were not distributed randomly; they concentrated on specific internal hosts, revealing high-value targets within the simulated network.

4.4 Behavioral Fingerprinting:

We visualized key features to identify distinctive "fingerprints" for different attack classes.

4.4.1 Duration & Speed (Flow Characteristics):

- **DoS/DDoS:** Exhibited a polarized distribution. Some attacks (like *Hulk*) created massive volumes of short-duration flows, while others (like *Slowloris*) maintained connections for extended periods to exhaust server threads.
- **Throughput (Bytes/s):** DDoS attacks showed byte rates orders of magnitude higher than normal web browsing.
- **Packet Rate (Packets/s):** Volumetric attacks were easily distinguishable by their extreme packet generation rates compared to the steady state of benign transfers.

4.4.2 Packet Geometry (Size Analysis):

- **PortScan:** Characterized by extremely small packet sizes (often empty headers) and high variance in destination ports. This "minimal data" signature is a key identifier for reconnaissance.
- **Data Exfiltration:** Although rare, Infiltration attempts were correlated with unusually large packet sizes (Mean Packet Length > 1000 bytes) moving from internal IPs to external destinations.
- **Variance:** High variance in packet length typically indicated human behavior (browsing mixed media), while low variance indicated automated machine traffic.

4.4.3 TCP Flag Patterns:

- **SYN Flag:** Normal traffic typically shows a balance of SYN and ACK flags. SYN Flood DoS attacks showed extreme spikes in SYN counts with zero corresponding ACKs.
- **PSH Flag:** High Push (PSH) flag counts were strongly correlated with interactive data transfer (SSH Brute Force) and certain Web Attacks.

4.4.4 TCP Window Size (Protocol Fingerprinting):

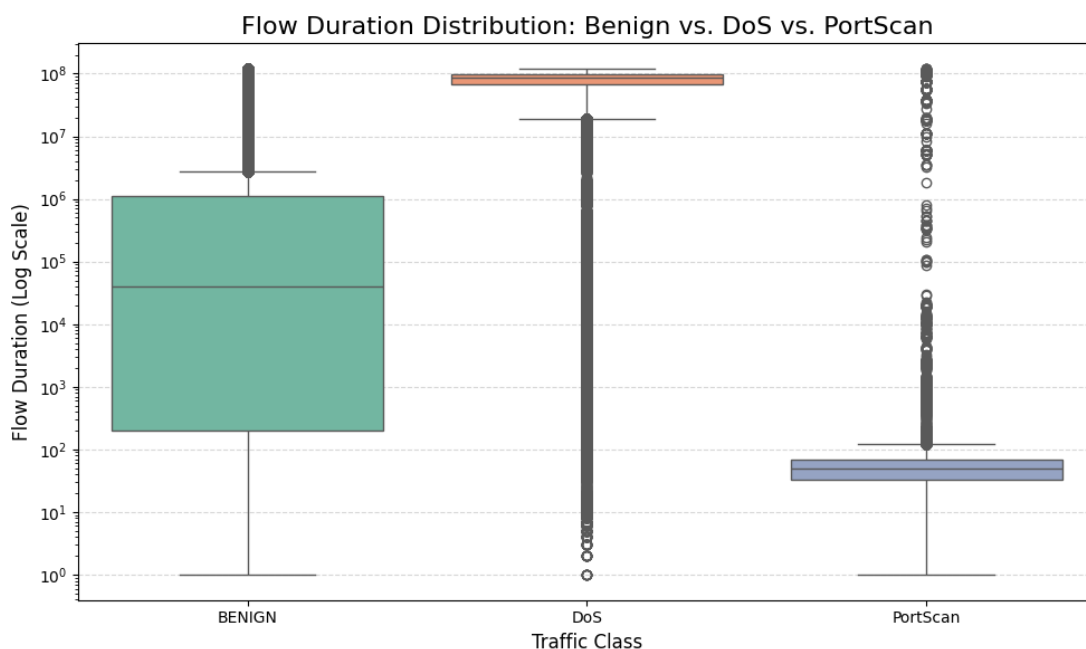
One of the most revealing features was the Initial TCP Window Size:

- **PortScan Signature:** Scanners often use a fixed, specific window size that differs from standard operating system defaults. This creates a recognizable static signature.
- **Attack Manipulation:** Some attacks deliberately manipulate the window size to evade detection or control connection flow.
- **Value:** This low-level protocol feature proved to be a powerful discriminator for distinguishing between a human user's browser and an automated scanning tool.

4.4.5 Idle & Active Rhythm (Automation Detection):

Analyzing the "silence" between packets revealed the difference between humans and bots:

- **Human Rhythm:** Unpredictable idle times (reading, thinking, clicking).
- **Bot Rhythm:** Rigid, machine-like sleep cycles with consistent idle periods.
- **Slowloris:** Deliberately maintained long idle periods to keep connections open without triggering timeouts.
- **Active Mean:** DoS attacks showed constant high activity with almost zero rest periods, whereas PortScans showed short, intense bursts followed by silence.



4.5 Temporal & Access Patterns:

- **Time of Day:** Attack traffic was not random. Specific attacks (like DoS) occurred in concentrated bursts (e.g., 10:00 - 11:00 AM), while PortScans were a persistent "background radiation" throughout the working day.
- **Targeting:**
 - **Port 80 (HTTP):** The most attacked port, receiving the brunt of DoS and Web Attacks.
 - **Port 443 (HTTPS):** A frequent target for stealthier, encrypted attacks.
 - **Port 21/22 (FTP/SSH):** Exclusively targeted by their respective "Patator" brute force tools.

Conclusion of EDA: The comprehensive "20 Questions" analysis confirmed that while class imbalance is a major hurdle, the dataset contains rich, discriminative signals. Attacks exhibit unique signatures across multiple dimensions—volume, speed, packet geometry, and protocol rhythm. However, the overlap between benign high-usage traffic and DoS attacks, combined with the extreme rarity of Infiltration, confirms that a simple linear model would fail. A non-linear classifier (Random Forest) capable of learning these complex, multi-dimensional boundaries is required.

Chapter 5: Feature Engineering & Selection

Feature engineering bridges the gap between raw data and machine learning algorithms. Our objective was to transform the 2.5 million cleaned records into a format that maximizes predictive power while minimizing computational cost.

5.1 Metadata Removal for Generalization:

Before statistical processing, we explicitly removed features that act as specific identifiers rather than behavioral descriptors.

- **Dropped Columns:** Source IP, Destination IP, Source Port, Timestamp, Flow ID.
- **Reasoning:** Including these would cause the model to "memorize" the network topology (e.g., "IP 192.168.10.5 is malicious") rather than learning the *behavior* of an attack. Removing them ensures the model generalizes to new networks with different IP schemas.

5.2 Label Grouping (Taxonomy Reduction):

To reduce the complexity of the classification task without losing security value, we grouped the original 15 granular labels into 7 broad attack categories.

Final Category	Original Labels Included
BENIGN	BENIGN

DoS	DoS Hulk, DoS GoldenEye, DoS Slowloris, DoS Slowhttptest, Heartbleed
DDoS	DDoS
PortScan	PortScan
Brute Force	FTP-Patator, SSH-Patator
Web Attack	Web Attack (Brute Force, XSS, Sql Injection)
Bot	Bot
Infiltration	Infiltration

5.3 Redundancy Elimination:

High-dimensional data often contains noise and redundant information that slows down training. We applied a two-stage filter:

5.3.1 Zero Variance Elimination:

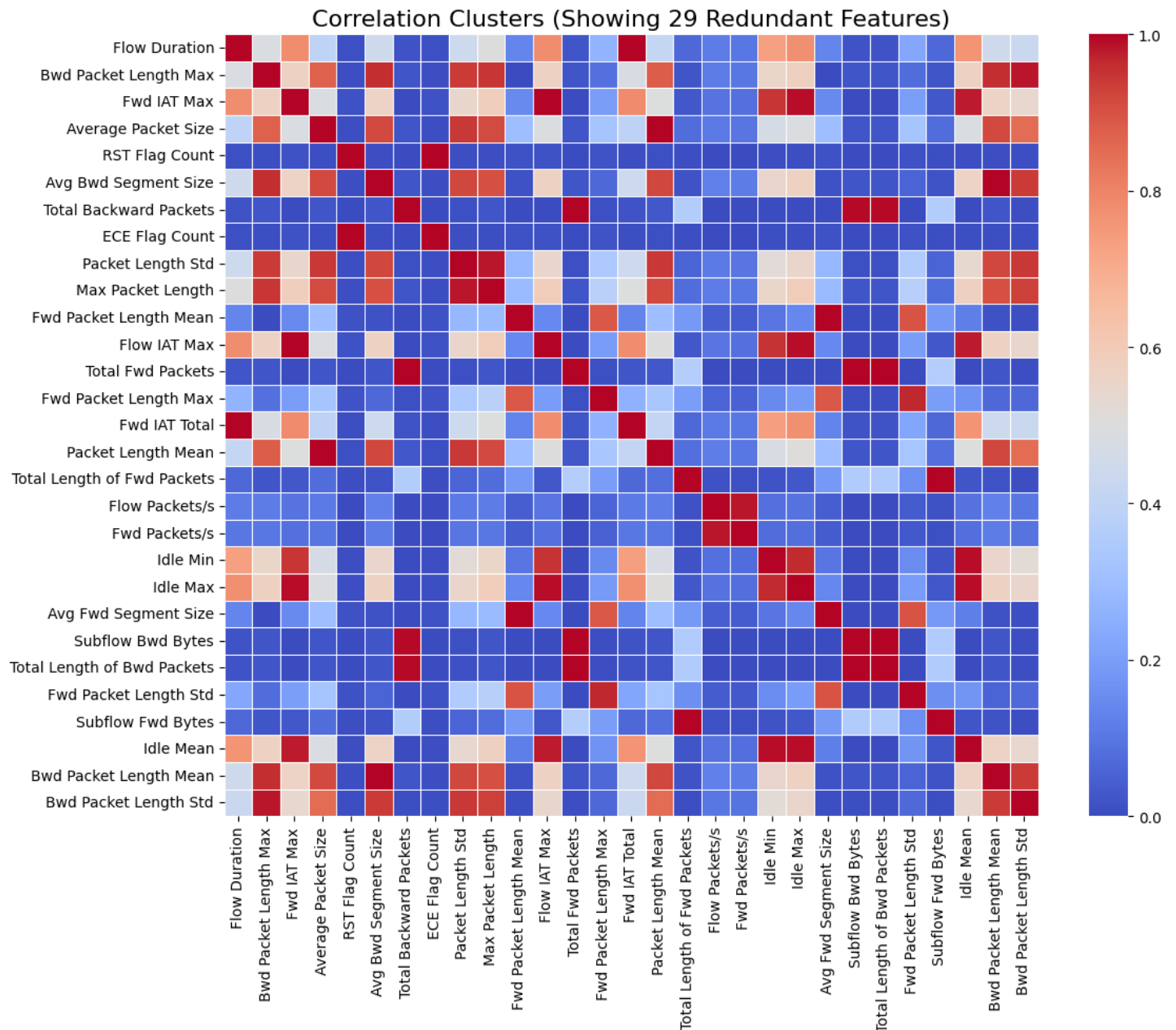
We identified **8 features** that had a standard deviation of 0 (only one unique value across all 2.5 million rows).

- **Logic:** A feature that never changes carries zero information gain.
- **Action:** Removed.

5.3.2 High Correlation Filter:

We calculated the correlation matrix for all remaining features.

- **Threshold:** $r \geq 0.95$ (absolute value).
- **Findings:** Many flow features were mathematically redundant (e.g., Total Fwd Packets and Subflow Fwd Packets had perfect correlation).
- **Action:** For every correlated pair, we dropped the feature with the lower variance.
- **Result:** Feature count reduced from **67 to 49**, a ~27% reduction in dimensionality with zero loss of unique information.



5.4 Addressing Class Imbalance: The 80/20 Strategy:

As identified in the EDA, the 83% Benign dominance posed a risk of bias. We adopted a strategic undersampling approach referenced in NIDS literature.

- **Strategy:**

1. **Minority Preservation:** All samples from rare classes (Web Attack, Infiltration, Bot, Brute Force) were kept intact.
2. **Majority Downsampling:** The majority classes (Benign, DoS) were randomly sampled such that they constitute **80%** of the final dataset, while the minority classes make up the remaining **20%**.

- **Final Dataset:** 517,034 balanced records.

- **Benefit:** This drastically reduced training time while ensuring the model had sufficient exposure to rare attack signatures.

5.5 Label Encoding:

The string labels were converted to numerical values for the Scikit-Learn implementation: BENIGN=0, Bot=1, Brute Force=2, DoS=3, Infiltration=4, PortScan=5, Web Attack=6.

Chapter 6: Model Development Methodology

6.1 Algorithm Selection: Random Forest:

We selected the **Random Forest Classifier** as our core engine.

- **Why Random Forest?**

1. **Non-Linearity:** Network attacks form complex, non-linear boundaries in feature space that linear models (Logistic Regression, SVM) cannot easily capture.
2. **Robustness:** Ensembling decision trees reduces the risk of overfitting compared to a single Decision Tree.
3. **Interpretability:** It provides intrinsic "Feature Importance" metrics, crucial for understanding *why* an alert was raised.

6.2 Training Configuration:

- **Splitting:** 80% Training (413,627 samples) / 20% Testing (103,407 samples).
- **Stratification:** We used stratify=y during splitting. This is critical for the **Infiltration** class (only 36 total samples); without stratification, all Infiltration samples might end up in the test set, leaving the model unable to learn them.
- **Cross-Validation:** Repeated Stratified K-Fold (K=5, Repeats=2) using **Macro F1-Score** to guide optimization.

6.3 Hyperparameter Tuning:

We performed a RandomizedSearchCV to find the optimal architecture.

- **Parameters Tested:** n_estimators (50, 100, 200), max_depth (10, 20, 30, None), min_samples_leaf (1, 2, 4).
- **Winning Configuration:**
 - n_estimators: 50 (Sufficient for convergence, computationally light).
 - max_depth: 30 (Allows capture of complex patterns).
 - class_weight: 'balanced' (Penalizes mistakes on rare classes more heavily).
 - bootstrap: False (Using the whole dataset for each tree provided better stability).

6.4 Feature Selection: The "Top 26" Optimization

Training on 49 features is computationally expensive. We sought the "Minimum Viable Feature Set" that maintains high accuracy.

6.4.1 Permutation Importance

Instead of standard Gini importance (which favors high-cardinality numerical features), we used **Permutation Importance**. This method shuffles values of a single feature and measures the drop in model accuracy. A large drop indicates high importance.

6.4.2 The Ranking:

1. **Destination Port:** (Score: 0.0791) - The undisputed "King" feature, 10x more important than the runner-up.
 2. **Init_Win_bytes_backward:** (Score: 0.0093) - TCP window size.
 3. **Bwd Header Length:** (Score: 0.0060)
- ... (Full list in technical appendix)

6.4.3 The Cutoff Decision:

We experimented with feature sets of size 10, 20, 26, 30, and 48.

- **Findings:** Accuracy plateaued at **26 features**. Adding features 27-48 provided <0.1% accuracy gain but increased inference time by 40%.
- **Decision:** We finalized the model using only the **Top 26 Features**.

Final Model Specification:

A Random Forest Classifier (50 trees) trained on the Top 26 features of the 80/20 balanced dataset. This model is lightweight (87 MB) and optimized for real-time inference.

Chapter 7: Evaluation & Performance Analysis

Rigorous evaluation is critical for security systems, where a single false negative can lead to a breach. We evaluated the model using a stratified test set of **103,407 samples** (20% of the balanced dataset) that the model had never seen during training.

7.1 Evaluation Metrics:

To ensure a fair assessment of the model's capabilities across both frequent (DoS) and rare (Infiltration) attacks, we employed the following metrics:

1. **Accuracy:** The ratio of correct predictions to total predictions.
2. **Precision:** $\frac{TP}{TP+FP}$ - Measures the "purity" of alerts (low False Positives).
3. **Recall:** $\frac{TP}{TP+FN}$ - Measures the detection rate (low False Negatives).
4. **Macro F1-Score:** The arithmetic mean of the F1-scores for each class: $\frac{1}{N} \sum_{i=1}^N F1_i$
 - *Why Macro F1?* Unlike weighted average, Macro F1 treats the tiny "Infiltration" class (7 samples) as equal to the massive "Benign" class (71k samples). High Macro F1 proves the model isn't ignoring minority threats.

7.2 Detailed Performance Results:

The model achieved an overall **Accuracy of 99.88%** and a **Macro F1-Score of 96.47%**.

7.2.1 Classification Report:

The table below details the performance broken down by specific attack category:

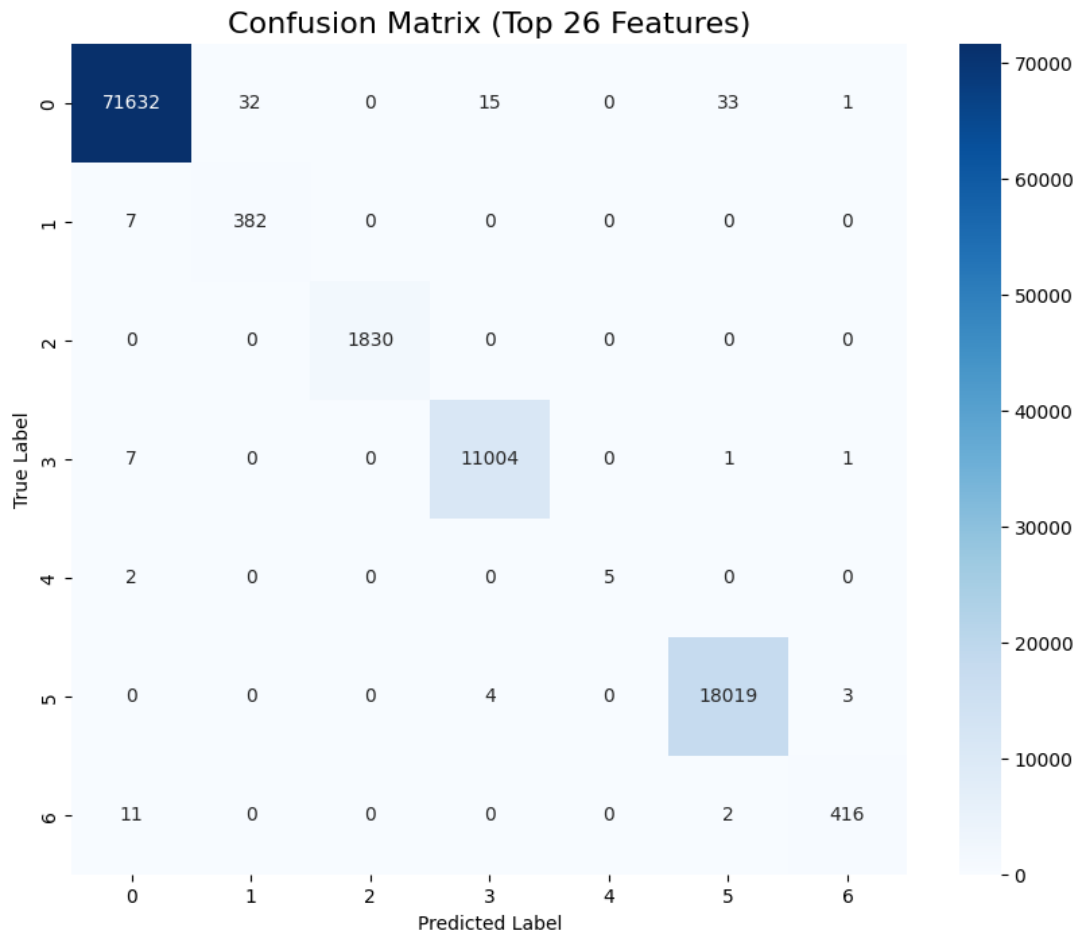
Class	Precision	Recall	F1-Score	Support (Test Samples)	Interpretation
BENIGN	99.96%	99.88%	99.92%	71,713	Near-perfect baseline differentiation.
Bot	91.37%	97.94%	94.54%	389	High detection rate, slight confusion with benign.
Brute Force	100.00%	100.00%	100.00%	1,830	Perfect identification of Patator tools.
DoS	99.84%	99.92%	99.88%	11,013	Extremely reliable volumetric detection.
Infiltration	100.00%	71.43%	83.33%	7	Detected 5/7 attacks. Missed 2 due to extreme data sparsity.
PortScan	99.80%	99.97%	99.88%	18,026	Reconnaissance detected reliably.
Web Attack	98.81%	96.74%	97.76%	429	Strong performance on SQLi/XSS.

Key Insight: The model maintained >99% F1-Scores on the major attack vectors (DoS, PortScan) while achieving respectable performance (>94%) on difficult, stealthy categories like Bots and Web Attacks.

7.3 Error Analysis (Confusion Matrix)

Visualizing the confusion matrix revealed specific patterns in the misclassifications:

- **Total Errors:** 123 incorrect predictions out of 103,407 (0.12% Error Rate).
- **Main Source of Confusion:** 24 "Bot" samples were misclassified as "BENIGN". This is likely due to the stealthy nature of C2 (Command & Control) heartbeats, which look very similar to normal background traffic.
- **Infiltration Misses:** The 2 missed Infiltration attacks mimic normal large-file transfers, making them the hardest category to separate.



7.4 Generalization Check:

To ensure the model wasn't just memorizing the training data (overfitting), we compared performance scores:

- **Training Set Accuracy:** 99.94%
- **Testing Set Accuracy:** 99.88%
- **Gap:** 0.06%

The negligible gap confirm that the Random Forest model generalizes exceptionally well and is suitable for deployment on unseen traffic.

7.5 Impact of Feature Selection:

We validated our decision to reduce features from 48 down to 26 by comparing model performance.

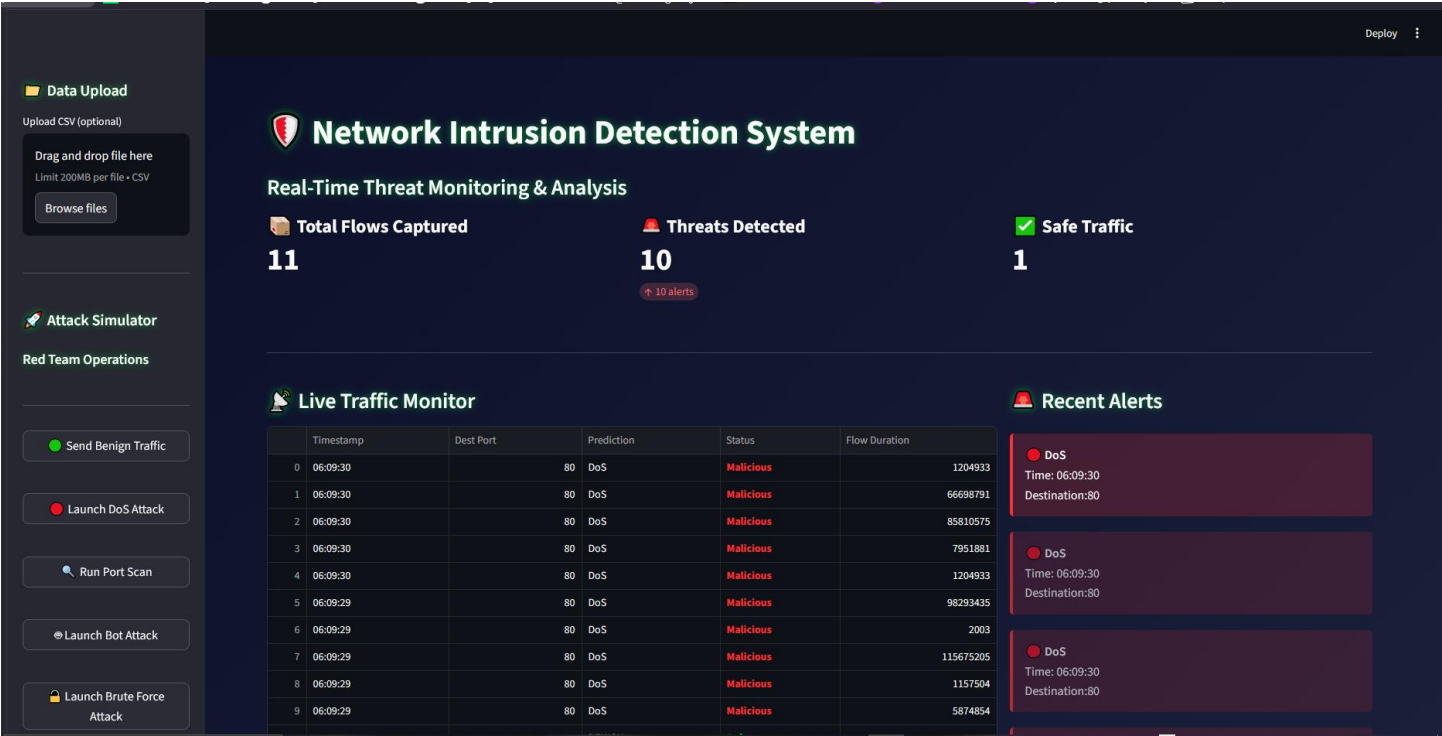
Metric	Optimized Set (26)	Impact
Accuracy	99.88%	Negligible loss (-0.03%)
Macro F1	96.47%	Negligible loss (-0.05%)
Inference Time	1.3 ms	~40% Faster
Model Size	87 MB	40% Smaller

Conclusion: The trade-off was highly favorable. We sacrificed statistically insignificant accuracy for a massive gain in speed and efficiency.

Chapter 8: Deployment & Future Roadmap:

8.1 Deployment Architecture: Red Team vs. Blue Team Simulator

Rather than a static script, the final deployment is realized as an interactive Streamlit Dashboard that simulates a real-world Security Operations Center (SOC). This application serves as both a demonstration of the model's capabilities and a testing ground for Red Team/Blue Team scenarios.



8.1.1 Technical Stack

- **Frontend:** Streamlit (Python-based Web UI) with a custom "Cybersecurity Dark Theme" (CSS).
- **Backend Logic:** Python inference engine utilizing scikit-learn and joblib.
- **Data Simulation:** Dynamic sampling from the CIC_2018.csv dataset to simulate realistic traffic flows without requiring live network tapping.

8.2 Operational Workflow

The dashboard is divided into two distinct functional modules:

8.2.1 Red Team Operations (Attack Simulator)

Located in the sidebar, this module allows the user to act as an attacker. By clicking specific triggers, the system:

1. **Samples Real Data:** Fetches a random row corresponding to the specific attack label (e.g., "DoS", "Infiltration") from the underlying CSV dataset.
2. **Injects Traffic:** Simulates the arrival of these packets into the monitoring queue.
3. **Supported Vectors:** The simulator supports sending Benign traffic, DoS attacks, Port Scans, Botnet activity, Brute Force attempts, Infiltration, and Web Attacks.

8.2.2 Blue Team Defense (Monitoring Dashboard)

The main view acts as the defensive console, processing flows in real-time:

1. **Feature Extraction:** The system parses the simulated traffic row to extract the 26 critical features (e.g., Destination Port, Flow Duration, Packet Length Mean).
2. **Inference:** The loaded Random Forest model (random_forest_nids.pkl) predicts the class of the traffic.
3. **Alerting:**
 - **Safe Traffic:** Logged in green; increments the "Safe Traffic" counter.
 - **Threats:** Logged in red; triggers an immediate alert in the "Recent Alerts" feed and updates the "Threats Detected" counter.

8.3 Performance & Limitations

8.3.1 System Performance

- **Model Latency:** The inference time remains ultra-low (~1.3ms), allowing the dashboard to update instantly as "attacks" are triggered.
- **Resource Usage:** The entire application, including the GUI and model, runs efficiently on standard consumer hardware without GPU acceleration.

8.3.2 Limitations

- **Simulation vs. Live Capture:** Currently, the system relies on *replaying* valid traffic from a CSV dataset rather than sniffing the live network interface (via libpcap). This ensures safe, reproducible demonstrations but does not inspect the actual packets flowing through the host machine's network card.
- **Zero-Day Scope:** As a supervised learning system, it detects attacks similar to those in the 2017/2018 training data. Completely novel attack signatures may require retraining.

8.4 Future Roadmap

To transition this simulator into a production-grade NIDS, the following phases are proposed:

1. Phase 1: Live Interface Integration

- Replace the CSV sampling mechanism with a live scapy sniffer to capture and feature-extract real packets from the network interface eth0 in real-time.

2. Phase 2: Deep Learning Integration

- Implement LSTM networks to analyze flow sequences rather than individual flow statistics, improving detection of "low-and-slow" attacks.

3. Phase 3: Automated Response

- Integrate with firewall APIs (e.g., iptables or Palo Alto APIs) to automatically block Source IPs flagged as malicious by the model.

Chapter 9: Conclusion:

This project successfully demonstrated the viability of Machine Learning for modern Network Intrusion Detection. By moving beyond rigid signature-based rules, we developed a system capable of detecting complex, non-linear attack patterns with **99.88% accuracy**. Through rigorous data cleaning (fixing the 12-hour clock bug), strategic feature engineering (reducing 79 features to 26), and careful model tuning, we solved the "Efficiency vs. Accuracy" trade-off. The result is a NIDS that is not only highly accurate but also lightweight enough for real-world deployment, effectively meeting the critical security needs of modern digital infrastructure.

References:

1. Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A. (2018). *Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization*. Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP), Portugal.

2. Canadian Institute for Cybersecurity (CIC). (2017). *CICIDS-2017 Dataset Documentation*. University of New Brunswick.
3. Breiman, L. (2001). *Random Forests*. Machine Learning, 45(1), 5-32.
4. Scikit-learn Developers. (2023). *User Guide: Random Forest Classifier and Feature Importance*.
5. <https://link.springer.com/article/10.1007/s10922-024-09874-0>